HTTP Requests and Responses

Learning Objective: Explain GET and POST methods and distinguish between 2xx, 4xx, and 5xx response status categories.

Target Level: Beginner	**Difficulty:** Intermediate	**Est. Duration:** 45 mins (Max: 45m)	**Status:** IN_REVIEW

1. Introduction (3 mins)

HTTP (Hypertext Transfer Protocol) is the foundation of data communication on the web. Every time you load a webpage, submit a form, or use a mobile app, your client sends HTTP requests and receives HTTP responses. Understanding how GET and POST methods work—and how to interpret 2xx, 4xx, and 5xx status codes—is essential for debugging APIs and building reliable client-server applications. This lesson focuses on practical API debugging practices so you can quickly identify whether an issue lies with the client request or the server response. [RFC 9110: HTTP Semantics] [MDN Web Docs: HTTP request methods & response status codes]

[Applied QC Fix - Factual]:
403 Forbidden: The server understood the request but refuses to authorize it. This may occur whether or not the client is authenticated.

2. Prerequisites (3 mins)

Before starting, you should be familiar with:\n- Basic client-server architecture (browser/client vs. server)\n- What a URL is (scheme, host, path)\n- Basic JSON structure (key-value pairs)\n- Opening browser Developer Tools (F12 or right-click → Inspect)\nNo prior networking or backend programming experience is required.

3. Key Concepts (6 mins)

HTTP Request: A message sent by a client to a server asking for a resource or action.\n**HTTP Response**: A message sent by the server back to the client containing status information and optional data.\n**GET**: Requests a representation of a specified resource. It should be safe and idempotent (no side effects, repeatable). [RFC 9110: HTTP Semantics]\n**POST**: Submits an entity to the specified resource, often causing a state change or side effect (e.g., creating a record). [RFC 9110: HTTP Semantics]\n**Status Code Classes**:\n- **2xx Success**: The request was received, understood, and accepted.\n- **4xx Client Error**: The request contains bad syntax or cannot be fulfilled due to client-side issues.\n- **5xx Server Error**: The server failed to fulfill a valid request.\n**Headers**: Metadata sent with requests and responses (e.g., Content-Type, Authorization). [MDN Web Docs: HTTP request methods & response status codes]

4. Detailed Explanations (10 mins)

GET vs POST\n**GET** is used to retrieve data. Parameters are typically sent in the URL query string (e.g., `/search?q=api`). GET requests should not modify server state and are idempotent—making the same GET request multiple times produces the same result. [RFC 9110: HTTP Semantics]\n\n**POST** is used to send data to the server, usually in the request body. Common uses include creating a new user, submitting a form, or uploading a file. POST is not idempotent by default; repeating a POST may create duplicate resources.\n\n## Status Code Categories for Debugging\n- **200 OK**: Standard success response. The body usually contains the requested resource.\n- **201 Created**: Common after a successful POST; the resource was created.\n- **204 No Content**: Success, but no body is returned.\n- **400 Bad Request**: The server could not understand the request due to invalid syntax (e.g., malformed JSON).\n- **401 Unauthorized**: Authentication is required or failed.\n- **403 Forbidden**: The client is authenticated but lacks permission.\n- **404 Not Found**: The requested resource does not exist.\n- **500 Internal Server Error**: A generic server-side failure.\n- **502 Bad Gateway / 503 Service Unavailable**: The server is temporarily overloaded or down.\n\n## API Debugging Workflow\n1. Reproduce the request in browser DevTools → Network tab.\n2. Check the status code: 2xx means success; 4xx means fix the client request; 5xx means contact the server team.\n3. Inspect request headers (Content-Type, Authorization) and payload.\n4. Inspect response headers and body for error messages.\n5. Use tools like `curl` or Postman to isolate the issue outside the browser. [MDN Web Docs: HTTP request methods & response status codes] [HTTP: The Definitive Guide]

5. Practical Examples (7 mins)

Example 1: GET Request\n**Request:**\n```\nGET /api/users/123 HTTP/1.1\nHost: api.example.com\nAccept: application/json\n```\n**Response (200 OK):**\n```\nHTTP/1.1 200 OK\nContent-Type: application/json\n\n{"id": 123, "name": "Alice", "role": "developer"}\n```\n\n### Example 2: POST Request\n**Request:**\n```\nPOST /api/users HTTP/1.1\nHost: api.example.com\nContent-Type: application/json\nAuthorization: Bearer <token>\n\n{"name": "Bob", "email": "bob@example.com"}\n```\n**Response (201 Created):**\n```\nHTTP/1.1 201 Created\nContent-Type: application/json\nLocation: /api/users/124\n\n{"id": 124, "name": "Bob", "email": "bob@example.com"}\n```\n\n### Example 3: Debugging a 400 Error\nIf you send invalid JSON to a POST endpoint, the server may respond:\n```\nHTTP/1.1 400 Bad Request\nContent-Type: application/json\n\n{"error": "Invalid JSON payload"}\n```\nThis tells you to fix the client-side request body before retrying. [MDN Web Docs: HTTP request methods & response status codes]

6. Real-World Applications (5 mins)

- **REST APIs**: Frontend applications call backend endpoints using GET to fetch data and POST to create or update records.\n- **Web Forms**: Traditional HTML forms use GET for searches and POST for submissions (login, registration).\n- **Mobile Apps**: Apps communicate with backend services over HTTPS using the same HTTP semantics.\n- **Production Debugging**: Site reliability engineers inspect 5xx spikes to detect outages, while frontend developers resolve 4xx errors caused by malformed requests or expired tokens. [HTTP: The Definitive Guide] [RFC 9110: HTTP Semantics]

7. Hands-on Activities (6 mins)

Activity: Inspect Live HTTP Traffic\n1. Open your browser and go to https://jsonplaceholder.typicode.com/posts/1\n2. Press F12 → Network tab → Reload the page.\n3. Click the first request (GET /posts/1). Observe:\n - Request

method and URL\n - Response status code (should be 200)\n - Response body (JSON)\n4. Open https://jsonplaceholder.typicode.com/posts in a new tab. Use the browser console to send a POST:\n ```javascript\n fetch('https://jsonplaceholder.typicode.com/posts', {\n method: 'POST',\n body: JSON.stringify({title: 'test', body: 'hello', userId: 1}),\n headers: { 'Content-Type': 'application/json' }\n })\n ```\n5. Check the Network tab for the POST request. Note the status code (201) and response.\n6. Deliberately send invalid JSON in the console and observe a 400 or 500 response. Practice identifying whether the error is client-side (4xx) or server-side (5xx). [MDN Web Docs: HTTP request methods & response status codes]

8. Knowledge Check (3 mins)

Question 1\nYou send a GET request to `/api/products` and receive a **200 OK** response. What does this indicate?\n**Answer:** The request succeeded and the server returned the requested resource. It is a 2xx success code.\n\n### Question 2\nA POST request to create a user returns **400 Bad Request**. Is this a client or server issue, and what should you check first?\n**Answer:** This is a client-side (4xx) issue. Check the request body for malformed JSON, missing required fields, or invalid data types.\n\n### Question 3\nWhich HTTP method is generally used to create a new resource on the server?\n**Answer:** POST. GET is for retrieving data; POST is for submitting data that may change server state. [RFC 9110: HTTP Semantics]\n\n### Question 4\nYou see a **503 Service Unavailable** error. Should you modify your client code?\n**Answer:** No. 5xx errors indicate a server-side problem. The client should retry later or alert the backend team.

9. Summary (2 mins)

You now understand the core HTTP request-response cycle. GET retrieves resources safely and idempotently, while POST submits data to create or modify resources. Status codes are grouped into classes: 2xx means success, 4xx means the client request needs fixing, and 5xx means the server encountered an error. By applying the debugging workflow—inspecting status codes, headers, and bodies in browser DevTools—you can quickly isolate API issues and determine whether they originate on the client or server. [RFC 9110: HTTP Semantics] [MDN Web Docs: HTTP request methods & response status codes]

10. Additional Resources & References

- **YouTube**: [HTTP Crash Course & Status Codes Explained](https://www.youtube.com/watch?v=iYM2zFP3Zn0) by Traversy Media — visual walkthrough of HTTP methods and status codes.\n- **Book**: *HTTP: The Definitive Guide* by David Gourley and Brian Totty (O'Reilly Media) — comprehensive textbook reference on HTTP architecture, headers, and status codes.\n- **Documentation**: [MDN Web Docs: HTTP](https://developer.mozilla.org/en-US/docs/Web/HTTP) — authoritative reference for request methods and response codes.\n- **Specification**: [RFC 9110: HTTP Semantics](https://www.rfc-editor.org/rfc/rfc9110.html) — the formal IETF standard defining HTTP methods and status code classes.